

Ask the Mutants: Mutating Faulty Programs for Fault Localization

Seokhyeon Moon, Yunho Kim, Moonzoo Kim

CS Dept. KAIST, South Korea

{seokhyeon.moon, kimyunho}@kaist.ac.kr, moonzoo@cs.kaist.ac.kr

Shin Yoo

CS Dept. University College London, UK

shin.yoo@ucl.ac.uk

Abstract—We present MUSE (MUTation-baSEd fault localization technique), a new fault localization technique based on mutation analysis. A key idea of MUSE is to identify a faulty statement by utilizing different characteristics of two groups of mutants—one that mutates a faulty statement and the other that mutates a correct statement. We also propose a new evaluation metric for fault localization techniques based on information theory, called Locality Information Loss (LIL): it can measure the aptitude of a localization technique for automated fault repair systems as well as human debuggers. The empirical evaluation using 14 faulty versions of the five real-world programs shows that MUSE localizes a fault after reviewing 7.4 statements on average, which is about 25 times more precise than the state-of-the-art SBFL technique Op2.

I. INTRODUCTION

Fault Localization (FL) is an expensive phase in the whole debugging activity because it usually takes human effort to understand the complex internal logic of the Program Under Test (PUT) and reason about the differences between passing and failing test runs. As a result, automated fault localization techniques have been widely studied.

One such technique is Spectrum-based Fault Localization (SBFL). It uses program spectra, i.e. summarized profile of test suite executions, to rank program statements according to their predicted risk of containing the fault. A developer, then, is to inspect PUT following the order of statements in the given ranking, in the hope that the faulty statement will be encountered near the top of the ranking [26].

SBFL has received much attention, with a heavy emphasis on designing new risk evaluation formulas [3, 14, 27, 31], but also on theoretical analysis of optimality and hierarchy between formulas [19, 28, 29]. However, it has also been criticized for their impractical accuracy and the unrealistic usage model that is the linear inspection of the ranking [22]. This is partly due to the limitations in the spectra data that SBFL techniques rely on. The program spectrum used by these techniques is simply a combination of the control flow of PUT and the results from test cases. Consequently, all statements in the same basic block share the same spectrum and, therefore, the same ranking. This often inflates the number of statements needed to be inspected before encountering the fault.

This paper presents a novel fault localization technique called MUSE, *MUTation-baSEd fault localization technique*, to overcome this problem. MUSE uses mutation analysis

to uniquely capture the relationship between individual program statements and the observed failures. It is free from the coercion of shared ranking from the block structure. The basic mutation testing is defined as artificial injection of syntactic faults [1]. However, we focus on what happens when we mutate an already faulty program and, particularly, the faulty program statement. Intuitively, since a faulty program can be repaired by modifying faulty statements, mutating (i.e., modifying) faulty statements will make more failed test cases pass than mutating correct statements. In contrast, mutating correct statements will make more passed test cases fail than mutating faulty statements. This is because mutating correct statements introduces new faulty statements in addition to the existing faulty statements in a PUT. These two observations form the basis of the design of our new metric for fault localization (Section II-A).

We also propose a new evaluation metric for fault localization techniques that is not tied to the ranking model. The traditional evaluation metric in SBFL literature is the Expense metric, which is the percentage of program statements the human developer needs to inspect before encountering the faulty one [18]. However, recent work showed that the Expense metric failed to account for the performance of the automated program repair tool that used various SBFL techniques to locate the fix: techniques proven to rank the faulty statement higher than others actually performed poorer when used in conjunction with a repair tool [23].

Our new evaluation metric, LIL (Locality Information Loss), actually measures the loss of information between the true locality of the fault and the predicted locality from a localization technique, using information theory. It can be applied to any fault localization technique (not just SBFL) and to describe localization of any number of faults.

Using both the traditional Expense metric and the LIL, we evaluate MUSE against 14 faulty versions of five real-world programs. The results show that MUSE is, on average, about 25 times more accurate than Op2 [19], the current state-of-the-art SBFL technique. In addition, MUSE ranks the faulty statement at the top of the suspiciousness ranking for seven out of 14 studied faults, and within the top three for another three faults. In addition, the newly introduced LIL metric also shows that MUSE can be highly accurate, as well as confirming the observation made by Qi et al. [23].

The contribution of this paper is as follows:

- The paper presents a novel fault localization technique called MUSE: *Mutation-based Fault Localization*. It utilizes mutation analysis to significantly improve the precision of fault localization.
- The paper proposes a new evaluation metric for fault localization techniques called *Locality Information Loss* (LIL) based on information theory. It is flexible enough to be applied to all types of fault localization techniques and can be easily applied to multiple faults scenarios.
- The paper presents an empirical evaluation of MUSE using five non-trivial real world programs. The results show that MUSE improves upon the best known SBFL technique by 25 times on average and ranks the faulty statement within the top 3 suspicious statements for 10 out of 14 subject program versions.

This paper is organized as follows. Section II describes the mutation-based fault localization technique to precisely localize a fault. Section III explains the new evaluation metric LIL based on information theory. Section IV shows the experiment setup for the empirical evaluation of the techniques on the subject programs. Section V explains the experiment results regarding the research questions and Section VI discusses the results. Section VII presents related work and Section VIII finally concludes with future work.

II. MUTATION-BASED FAULT LOCALIZATION

A. Intuitions

Consider a faulty program P whose execution with some test cases results in failures and we propose to mutate P . Let m_f be a mutant of P that mutates the faulty statement, and m_c one that mutates a correct statement. MUSE depends on the following two conjectures.

Conjecture 1: test cases that used to fail on P are more likely to pass on m_f than on m_c .

The first conjecture is based on the observation that m_f can only be one of the following three cases per test suite:

- 1) **Equivalent/dormant mutant** (i.e. mutants that syntactically change the program but not semantically), in which case the faulty statement remains faulty. Tests that failed on P should still fail on m_f .
- 2) **Non-equivalent and faulty**: while the new fault may or may not be identical to the original fault, we expect tests that have failed on P are still more likely to fail on m_f than to pass.
- 3) **Non-equivalent and not faulty**: in which case the fault is fixed by the mutation (with respect to the test suite concerned).

Note that mutating the faulty statement is more likely to cause the tests that failed on P to pass on m_f (case 3) than on m_c because a faulty program is usually fixed by modifying (i.e., mutating) a faulty statement, not a correct one. Therefore, the number of the failing test cases whose results change to pass will be larger for m_f than for m_c .

In contrast, mutating correct statements is not likely to make more test cases pass. Rather, we expect an opposite effect, which is as follows:

Conjecture 2: test cases that used to pass on P are more likely to fail on m_c than on m_f .

Similarly to the case of m_f , the second conjecture is based on an observation that m_c can be one of the following cases per test suite:

- 1) **Equivalent/dormant mutant**, in which case the statement remains correct. Tests that passed with P should still pass with m_c .
- 2) **Non-equivalent mutant**: by definition, a non-equivalent mutation on a correct statement introduces a fault, which is the original premise of mutation testing.

This second conjecture is based on the observation that a program is more easily broken by modifying (i.e., mutating) a correct statement than by modifying a faulty statement (case 2). Therefore, the number of the passing test cases whose results change to fail will be greater for m_c than m_f .

To summarize, mutating a faulty statement is more likely to cause more tests to pass than the average, whereas mutating a correct statement is more likely to cause more tests to fail than the average (the average case considers both correct and faulty statements). These conjectures provide the basis for our mutation-based fault localization technique.

B. Suspiciousness Metric of MUSE

Based on the conjectures, we now define the suspiciousness metric for MUSE, μ . For a statement s of P , let $f_P(s)$ be the set of tests that covered s and failed on P , and $p_P(s)$ be the set of tests that covered s and passed on P . With respect to a fixed set of mutation operators, let $mut(s) = \{m_1, \dots, m_k\}$ be the set of all mutants of P that mutates s with observed changes in test results (we use only non-dormant mutants since dormant mutants do not provide useful information to utilize the conjectures). After each mutation $m_i \in mut(s)$, let f_{m_i} and p_{m_i} be the set of failing and passing tests on m_i respectively (f_P and p_P defined on P similarly). Given a weight α , the metric μ is defined as follows:

$$\mu(s) = \frac{1}{|mut(s)|} \sum_{m \in mut(s)} \left(\frac{|f_P(s) \cap p_m|}{|f_P|} - \alpha \cdot \frac{|p_P(s) \cap f_m|}{|p_P|} \right) \quad (1)$$

The first term, $\frac{|f_P(s) \cap p_m|}{|f_P|}$, reflects the first conjecture: it is the proportion of tests that failed on P but now pass on a mutant m that mutates s over tests that failed on P . Similarly, the second term, $\frac{|p_P(s) \cap f_m|}{|p_P|}$, reflects the second conjecture, being the proportion of tests that passed on P but now fail on a mutant m that mutates s over tests that passed on P . When averaged over $mut(s)$, they become the probability of test result change per mutant, from failing to passing and vice versa respectively.

		Coverage of Test Cases (x, y)							Jaccard		Ochiai		Op2	
<pre>int max; void setmax(int x, int y){ max = -x; //should be 'max = x;'</pre>		TC ₁ (3,1)	TC ₂ (5,-4)	TC ₃ (0,-4)	TC ₄ (0,7)	TC ₅ (-1,3)	$ f_P(s) $	$ p_P(s) $	Susp.	Rank	Susp.	Rank	Susp.	Rank
s ₁ :	max = -x; //should be 'max = x;'	●	●	●	●	●	2	3	0.40	5	0.63	5	1.25	5
s ₂ :	if(max < y){	●	●	●	●	●	2	3	0.40	5	0.63	5	1.25	5
s ₃ :	max = y;	●	●		●	●	2	2	0.50	2	0.71	2	1.50	2
s ₄ :	if(x*y<0)	●	●		●	●	2	2	0.50	2	0.71	2	1.50	2
s ₅ :	print(`diff.sign`);}		●			●	1	1	0.33	6	0.50	6	0.75	6
s ₆ :	print(max);}	●	●	●	●	●	2	3	0.40	5	0.63	5	1.25	5
Test Results		Fail	Fail	Pass	Pass	Pass								
		Test Result Changes							MUSE					
Statements		Mutants	TC ₁ (3,1)	TC ₂ (5,-4)	TC ₃ (0,-4)	TC ₄ (0,7)	TC ₅ (-1,3)	$\frac{ f_P(s) }{p_m }$	$\frac{ p_P(s) }{f_m }$	Suspiciousness		Rank		
s ₁ :	max = -x;	m1: max -= x-1; m2: max=x;	F→P	F→P	P→F			0 2	1 0	0.46		1		
s ₂ :	if(max < y){	m3: if(!(max<y)){ m4: if(max==y){	F→P		P→F	P→F	P→F	0 1	3 1	0.09		2		
s ₃ :	max = y;	m5: max = -y; m6: max = y+1;				P→F	P→F	0 0	2 2	-0.16		5		
s ₄ :	if(x*y<0){	m7:if(!(x*y<0)) m8:if(x/y<0)				P→F	P→F	0 0	2 1	-0.12		4		
s ₅ :	print(`diff.sign`);}	m9:return; m10;;					P→F	0 0	1 1	-0.08		3		
s ₆ :	print(max);}	m11:printf(0);} m12:;}			P→F	P→F	P→F	0 0	2 3	-0.20		6		

Figure 1: Example of how MUSE localizes a fault compared with different fault localization techniques

Intuitively, the first term correlates to the probability of s being the faulty statement (it increases the suspiciousness of s if mutating s causes failing tests to pass, i.e. increase the size of $f_P(s) \cap p_m$), whereas the second term correlates to the probability of s not being the faulty statement (it decreases the suspiciousness of s if mutating s causes passing tests to fail, i.e. increase the size of $p_P(s) \cap f_m$).

Since it is more likely that a passing test case on P will fail on m than a failing test case on P will pass on m (i.e., breaking a program is easier than correcting the program), we expect the average of the second term to be different from that of the first term. In order to balance the two terms, we use the weight α to adjust the average values of the two terms to be the same. Thus, when we subtract the weighted second term from the first term as in Equation 1, we get the baseline of value 0. For a faulty statement, the first term is likely to be larger and the second term is likely to be smaller than for a correct statement (we assign minimum suspiciousness to the statements that do not have a mutant).

To adjust the average of both terms, the value of α should be calculated as $\frac{f_{2p}}{|mut(P)| \cdot |f_P|} \cdot \frac{|mut(P)| \cdot |p_P|}{p_{2f}}$. Variable f_{2p} and p_{2f} denote the number of test result changes from failure to pass and vice versa between before and after all mutants of P , the set of which is $mut(P)$. Note that α can be calculated without *a priori* knowledge of the faulty statement and we can use other fault localization techniques if $\alpha=0$.

C. A Working Example

Figure 1 presents an example of how MUSE localizes a fault. The PUT is a function called `setmax()`, which sets a global variable `max` (initialized to 0) with `y` if `x < y`, or with `x` otherwise. Statement s_1 contains a fault, as

it should be `max=x`. Let us assume that we have five test cases ($tc1$ to $tc5$): the coverage of individual test cases are marked with black bullets (•). TC_1 and TC_2 fail because `setmax()` updates `max` with the smaller number, `y`. The remaining test cases pass. Thus, $|f_P| = 2$ and $|p_P| = 3$.

First, MUSE generates mutants by mutating only one statement at a time. For the sake of simplicity, here we assume that MUSE generates only two mutants per statement, resulting in a total of 12 mutants, $\{m_1, \dots, m_{12}\}$ (listed under the “Mutants” column of Figure 1). Test cases change their results after the mutation as noted in the middle column. For example, TC_1 , which used to fail, now passes on the two mutants, m_2 and m_4 .

Based on the changed results of the test cases, MUSE calculates α as $\frac{f_{2p}}{|mut(P)| \cdot |f_P|} \cdot \frac{|mut(P)| \cdot |p_P|}{p_{2f}} = \frac{3}{12 \cdot 2} \cdot \frac{12 \cdot 3}{19} = 0.24$ over 12 mutants ($|mut(P)| = 12$). Since there are three changes from failure to pass, $f_{2p} = 3$ (TC_1 and TC_2 on m_2 and TC_1 on m_4) while $|f_P| = 2$. Similarly, $p_{2f} = 19$ (see the changed results of TC_3 , TC_4 , and TC_5), while $|p_P| = 3$.

Using $\alpha = 0.24$, MUSE calculates the suspiciousness of s_1 as $\frac{1}{2} \cdot \{(0/2 - 0.24 \cdot 1/3) + (2/2 - 0.24 \cdot 0/3)\} = 0.46$, where $|f_P(s_1) \cap p_{m_1}| = 0$ and $|p_P(s_1) \cap f_{m_1}| = 1$ for m_1 and $|f_P(s_1) \cap p_{m_2}| = 2$ and $|p_P(s_1) \cap f_{m_2}| = 0$ for m_2 . MUSE calculates the suspiciousness scores of the other five statements as 0.09, -0.16, -0.12, -0.08, and -0.20. The suspiciousness of the s_1 is the highest (i.e., at the top of the ranking). In contrast, Jaccard [10], Ochiai [20], and Op2 [19] choose s_3 and s_4 as the most suspicious statements, while assigning the fifth rank to the actual faulty statement s_1 . The example shows that MUSE can precisely locate certain faults that the state-of-the-art SBFL techniques cannot.

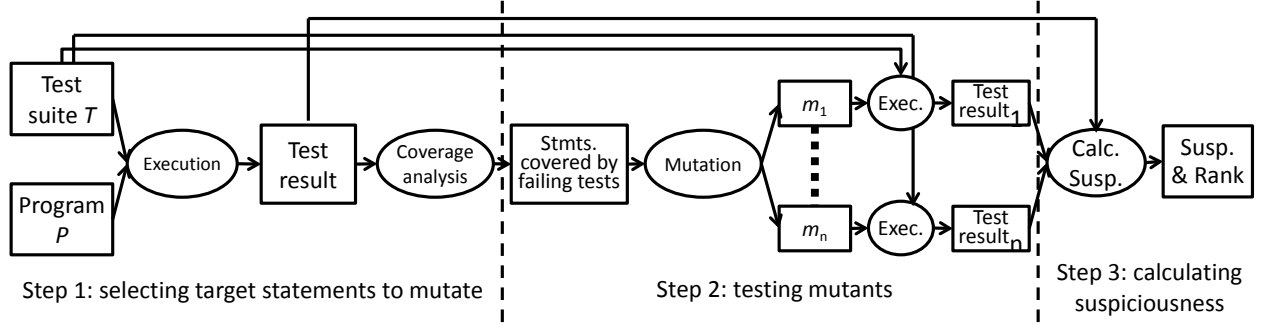


Figure 2: Framework of MUTation-baSEd fault localization technique (MUSE)

D. MUSE Framework

Figure 2 shows the framework of MUTation-baSEd fault localization technique (MUSE). There are three major stages: *selection* of statements to mutate, *testing* of the mutants, and *calculation* of the suspiciousness scores.

Step 1: MUSE receives a target program P and a test suite T . After executing T on P , MUSE selects the target statements, i.e. the statements of P that are executed by at least one failing test case in T . We focus on only these statements as those not covered by any failing tests, can be considered not faulty with respect to T .

Step 2: MUSE generates mutant versions of P by mutating each of the statements selected at Step 1. MUSE may generate multiple mutants from a single statement since one statement may contain multiple mutation points [8]. MUSE tests all generated mutants with T and records the results.

Step 3: MUSE compares the test results of T on P with the test results of T on all mutants. This produces the weight α , based on which MUSE calculates the suspiciousness of the target statements of P .

III. LIL: LOCALITY INFORMATION LOSS

The output of fault localization techniques can be consumed by either human developers or automated program repair techniques. Expense [18] metric measures the portion of program statements that need to be inspected by developers until the localization of the fault. It has been widely adopted as an evaluation metric for FL techniques [13, 19, 31] as well as a theoretical framework that showed hierarchies between SBFL techniques [28, 29]. However, the Expense metric has been criticised for being unrealistic to be used by a human developer directly [22].

In an attempt to evaluate the precision of SBFL techniques, Qi et al. [23] compared SBFL techniques by measuring the Number of Candidate Patches (NCP) generated by GenProg [25] automated program repair tool, with the given localization information.¹ Automated program repair techniques tend to bypass the ranking and directly use the

¹Essentially this measures the number of fitness evaluation for the Genetic Programming part of GenProg; hence the lower the NCP score is, the more efficient GenProg becomes, and in turn the more effective the given localization technique is.

suspiciousness scores of each statement as the probability of mutating the statement (expecting that mutating a highly suspicious statement is more likely to result in a potential fix) [6, 25]. An interesting empirical observation by Qi et al. [23] is that Jaccard [10] produced lower NCP than Op2 [19], despite having been proven to always produce a lower ranking for the faulty statement than Op2 [28]. This is due to the actual distribution of the suspiciousness score: while Op2 produced higher ranking for the faulty statement than Jaccard, it assigned almost equally high suspiciousness scores to some correct statements. On the other hand, Jaccard assigned much lower suspiciousness scores to correct statements, despite ranking the faulty statement slightly lower than Op2.

This illustrates that evaluation and theoretical analysis based on the linear ranking model is not applicable to automated program repair techniques. LIL metric can measure the aptitude of FL techniques for automated repair techniques as it measures the effectiveness of localization in terms of information loss rather than the behavioural cost of inspecting a ranking of statements. LIL metric essentially captures the essence of the entropy-based formulation of fault localization [32] in the form of an evaluation metric.

We propose a new evaluation metric that does not suffer from this discrepancy between two consumption models. Let S be the set of n statements of the Program Under Test, $\{s_1, \dots, s_n\}$, s_f , ($1 \leq f \leq n$) being the single faulty statement. Without losing generality, we assume that output of any fault localization technique τ can be normalized to $[0, 1]$. Now suppose that there exists an ideal fault localization technique, $\mathcal{L}$, that can always pinpoint s_f as follows:

$$\mathcal{L}(s_i) = \begin{cases} 1 & (s_i = s_f) \\ \epsilon & (0 < \epsilon \ll 1, s_i \in S, s_i \neq s_f) \end{cases} \quad (2)$$

Note that we can convert outputs of FL techniques that do not use suspiciousness scores in a similar way: if a technique τ simply reports a set C of m statements as candidate faulty statements, we can set $\tau(s_i) = \frac{1}{m}$ when $s_i \in C$ and $\tau(s_i) = \epsilon$ when $s_i \in S \setminus C$.

We now cast the fault localization problem in a probabilistic framework as in the previous work [32]. Since the *suspiciousness* score of a statement is supposed to correlate

to the likelihood of the statement containing the fault, we convert the suspiciousness score given by an FL technique, $\tau : S \rightarrow [0, 1]$, into the probability of any member of S containing the fault, $P_\tau(s)$, as follows:

$$P_\tau(s_i) = \frac{\tau(s_i)}{\sum_{i=1}^n \tau(s_i)}, (1 \leq i \leq n) \quad (3)$$

This converts suspiciousness scores given by any τ (including $\mathcal{L}$) into a probability distribution, P_τ . The metric we propose is the Kullback-Leibler divergence [16] of P_τ from $P_\mathcal{L}$, denoted as $D_{KL}(P_\mathcal{L}||P_\tau)$: it measures the information loss that happens when using P_τ instead of $P_\mathcal{L}$ and is calculated as follows:

$$D_{KL}(P_\mathcal{L}||P_\tau) = \sum_i \ln \frac{P_\mathcal{L}(s_i)}{P_\tau(s_i)} P_\mathcal{L}(s_i) \quad (4)$$

We call this as Locality Information Loss (LIL). Kullback-Leibler divergence between two given probability distribution P and Q requires the following: both P and Q should sum to 1, and $Q(s_i) = 0$ implies $P(s_i) = 0$. We satisfy the former by the normalization in Equation 3 and the latter by always substituting 0 with ϵ after normalizing τ^2 (because we cannot guarantee the implication in our application). When these properties are satisfied, $D_{KL}(P_\mathcal{L}||P_\tau)$ becomes 0 when $P_\mathcal{L}$ and P_τ are identical. As with the Expense metric, the lower the LIL value is the more accurate the FL technique is. Based on Information Theory, LIL has the following strengths compared to the Expense metric:

- **Expressiveness:** unlike the Expense metric that only concerns the actual faulty statement, LIL also reflects how well the suspiciousness of non-faulty statements have been suppressed by an FL technique. That is, LIL can be used to explain the results of Qi et al. [23] quantitatively.
- **Flexibility:** unlike the Expense metric that only concerns a single faulty statement, LIL can handle multiple locations of faults. For m faults (or for a fault that consists of m different locations), the distribution $P_\mathcal{L}$ will simply show not one but m spikes, each with $\frac{1}{m}$ as height.
- **Applicability:** Expense metric is tied to FL techniques that produce rankings, whereas LIL can be applied to any FL technique. If a technique assigns suspiciousness scores to statements, it can be converted into P_τ ; if a technique simply presents one or more statements as candidate fault location, P_τ can be formulated to have corresponding peaks.

IV. EXPERIMENTAL SETUP

We have designed the following three research questions to evaluate the effectiveness of MUSE in terms of the

² ϵ should be smaller than the smallest normalized non-zero suspiciousness score by τ .

Expense metric [18] and the LIL metric (Section III):

RQ1. Foundation: *How many test results change from failure to pass and vice versa between before and after on a mutant generated by mutating a faulty statement, compared with a mutant generated by mutating a correct one?*

RQ1 is to validate the conjectures in Section II-A, on which MUSE depends. If these conjectures are valid (i.e., more failing test cases become passing after mutating the faulty statement than a correct one, and more passing test cases become failing after mutating a correct statement than the faulty one), we can expect that MUSE will localize a fault precisely.

RQ2. Precision: *How precise is MUSE, compared with Jaccard, Ochiai, and Op2 in terms of the % of executed statements examined to localize a first fault?*

Precision in terms of the % of program statements to be examined is the traditional evaluation criteria for fault localization techniques. RQ2 evaluates MUSE with the Expense metric against the three widely studied SBFL techniques – Jaccard, Ochiai, and Op2. Op2 [19] is *proven* to perform well in Expense metric; Ochiai [20] performs closely to Op2, while Jaccard [10] shows good performance when used with automated program repair [23].

RQ3. Information Loss: *How precise is MUSE, compared with Jaccard, Ochiai, and Op2 in terms of the Locality Information Loss (LIL) metric?*

RQ3 evaluates the precision of MUSE with the LIL metric introduced in Section III against the three SBFL techniques (Jaccard, Ochiai, and Op2). The smaller the LIL value is, the more precise the FL technique is.

To answer the research questions, we performed a series of experiments by applying Jaccard, Ochiai, Op2, and MUSE to the 14 faulty versions in five real world C programs. The following subsections describe the details of the experiments.

A. Subject Programs

For the experiments, we used five non-trivial real-world programs including `flex` version 2.4.7, `grep` version 2.2, `gzip` version 1.1.2, `sed` version 1.18, and `space`, all of which are from the SIR benchmark suite [4].

Table I describes the target programs including their sizes in Lines of Code, the faulty versions used, and the numbers of failing and passing test cases for each program version/fault pair. From the base versions listed above, we randomly selected three faulty versions from each program except `grep` where a failure is detected only in two faulty versions by the used test suite. `grep v3` and `space v19` have multiple faults and the other versions have one fault per each version. The fault ID of each version is presented in Table I (For the rest of the paper, we refer to

Table I: Subject programs, their sizes in lines of code (LOC), and the number of failing and passing test cases

Subjects	Ver.	Fault	Size	$ f_P $	$ p_P $	Description
flex	v1	F_HD_1	12,423	2	40	Lexical Analyzer Generator
	v7	F_HD_7	12,423	1	41	
	v11	F_AA_3	12,423	20	22	
grep	v3	F_DG_4	12,653	5	175	Pattern Matcher
	v11	F_KP_2	12,653	177	22	
gzip	v2	F_KL_2	6,576	1	211	Compression Utility
	v5	F_KP_1	6,576	17	196	
	v13	F_KP_9	6,576	3	210	
sed	v1	F_AG_2	11,990	42	316	Stream Editor
	v3	F_AG_17	11,990	1	357	
	v5	F_AG_20	11,990	64	81	
space	v19	N/A	9,129	8	145	ADL Interpreter
	v21	N/A	9,126	1	152	
	v28	N/A	9,126	46	107	

these faulty versions with the term *version*).³ For *flex*, *grep*, and *space*, we used the coverage-adequate test suite provided by the SIR benchmark (*flex* and *grep* has only one coverage adequate test suite. For *space*, we randomly chose one coverage adequate test suite out of 1000 coverage-adequate test suites). For *gzip* and *sed*, we use the universe test suite, because the SIR benchmark does not provide a coverage-adequate test suite for the two programs. In addition, we excluded the test cases which caused a target program version to crash (e.g., segmentation fault), since *gcov* that we used to measure coverage information cannot record coverage information for such test cases.

B. Mutation and Fault Localization Setup

We use *gcov* to measure the statement coverage achieved by a given test case. Based on the coverage information, MUSE generates mutants of the PUT, each of which is obtained by mutating one statement that is covered by at least one failing test case. We use the Proteum mutation tool for the C language [17], which implements the mutation operators defined by Agrawal et al. [8]. To reduce the cost of the experiments, MUSE generates only one mutant for each mutation point of a target statement per mutation operator using the options provided by Proteum.⁴

We implemented MUSE, as well as Jaccard, Ochiai, and Op2, in 6,400 lines of C++ code. All experiments were

³MUSE does not assume that a fault lie in one statement because a partial fix of a multi-line spanning fault obtained by mutating one statement can still correct (or partially correct (i.e., make the target program pass with a subset of failing test cases)) the target program and provide important information to localize a fault.

⁴For example, `if(x+2>y+1)` has one mutation point (`>`) for ORRN (mutation operator on relational operator) and two points (2 and 1) for CCCR (mutation operator for constant to constant replacement) [8]. MUSE generates only one mutant like `if(x+2<y+1)` using ORRN and only `if(x+0>y+1)` and `if(x+2>y+0)` using CCCR. The selection of a mutant to generate using a mutation operator depends on the Proteum implementation.

Table II: The number of target statements, used mutants, and dormant mutants (those that do not change any test results) per subject

Subjects	Target Stmt.	Used Mutants	Dormant Mutants
flex v1	2,243	29,030	7,375
flex v7	2,209	28,575	7,411
flex v11	2,473	30,366	8,532
grep v3	1,364	18,127	10,201
grep v11	1,652	12,029	26,425
gzip v2	129	1,172	835
gzip v5	263	2,054	1,896
gzip v13	143	1,238	887
sed v1	1,694	13,215	4,813
sed v3	887	6,307	2,367
sed v5	1958	23,552	0
space v19	2,124	14,489	4,919
space v21	1,509	9,708	2,790
space v28	2,405	13,946	7,443
Average	1503.8	14557.7	6135.3

performed on 10 machines equipped with Intel i5 3.6Ghz CPUs and 8GB RAM running 64 bit Debian Linux 6.05.

V. RESULT OF THE EXPERIMENTS

A. Result of the Mutation

Table II shows the number of mutants generated per subject program version. On average, MUSE generates 20693.0 (=14557.7+6135.3) mutants per version and uses 14557.7 mutants, while discarding 6135.3 *dormant* mutants, i.e. those for which none of the test cases change their results, on average.⁵ This translates into an average of 9.7 mutants per considered target statement. The mutation and the subsequent testing of all mutant versions took 5 hours using the 10 machines while each of Jaccard, Ochiai, and Op2 took several minutes on one machine. Note that the mutation task of MUSE can be highly parallelized/distributed on thousands of machines (for example, utilizing Amazon EC2 cloud computing platform) and MUSE can localize a fault in several minutes since mutating a statement s_i is independent of mutating another statement s_j ($i \neq j$) and testing each mutant is also independent to each other.

B. Regarding RQ1: Validity of the Conjectures

Table III shows the numbers of the test cases whose results change on each mutant of the target programs. The second and the third columns show the average numbers of failing test cases on P which subsequently pass after mutating a correct statement (i.e. m_c), or a faulty statement (i.e. m_f), respectively. The fifth and the sixth columns show the average numbers of the passing test cases on P which subsequently fail on m_c and m_f respectively. For example, on average, out of the 17 failing test case of *gzip* v5,

⁵*sed* v5 has no dormant mutant because the fault of *sed* v5 is non-deterministic one (i.e., it dynamically allocates an smaller amount of memory than necessary through `malloc()`).

Table IV: Precision of Jaccard, Ochiai, Op2, and MUTation-baSEd fault localization technique (MUSE)

Subject Program	% of executed stmts examined				Rank of a faulty stmt				Locality Information Loss			
	Jaccard	Ochiai	Op2	MUSE	Jaccard	Ochiai	Op2	MUSE	Jaccard	Ochiai	Op2	MUSE
flex v1	49.48	45.04	32.01	0.04	1,371	1,248	887	1	8.33	7.89	7.68	1.28
flex v7	3.60	3.60	3.60	0.07	100	100	100	2	5.72	6.52	7.45	1.22
flex v11	19.76	19.54	13.51	0.04	547	541	374	1	7.39	7.49	7.40	1.59
grep v3	1.06	1.01	0.71	1.87	21	20	14	37	5.25	5.68	6.21	5.92
grep v11	3.44	3.44	3.44	1.60	58	58	58	27	5.43	6.20	5.46	7.19
gzip v2	2.14	2.14	2.14	0.07	31	31	31	1	5.18	4.62	6.24	1.66
gzip v5	1.83	1.83	1.83	0.07	26	26	26	1	4.45	4.73	5.27	1.88
gzip v13	1.03	1.03	1.03	0.07	15	15	15	1	3.12	3.65	5.71	0.70
sed v1	0.54	0.54	0.54	0.90	12	12	12	20	4.24	5.02	5.80	6.72
sed v3	2.56	2.56	2.56	0.13	57	57	57	3	6.14	5.92	6.40	2.66
sed v5	37.84	37.84	37.15	0.28	814	814	799	6	7.34	7.42	7.34	4.80
space v19	0.03	0.03	0.03	0.06	1	1	1	2	5.27	5.93	6.64	2.15
space v21	0.45	0.45	0.45	0.03	15	15	15	1	4.92	5.96	7.34	0.40
space v28	11.57	10.66	6.89	0.04	329	303	196	1	7.33	7.40	7.24	1.96
Average	9.67	9.27	7.56	0.38	242.64	231.50	184.64	7.43	5.72	6.03	6.58	2.87

Table III: The average numbers of the test cases whose results change on the mutants

Subject programs	# of Failing Tests that Pass after Mutating:			# of passing tests that fail after mutating:			α
	Correct Stmt.	Faulty Stmt.	(B)/(A)	Correct Stmt.	Faulty Stmt.	(C)/(D)	
	(A)	(B)		(C)	(D)		
flex v1	0.0002	1.2727	6155.6	15.7270	8.8182	1.8	0.0009
flex v7	0.0002	0.6667	2721.1	16.3644	0.0000	N/A	0.0007
flex v11	0.0026	14.2857	5421.3	5.1064	3.5714	1.4	0.0013
grep v3	0.1299	0.4792	3.7	30.7825	8.0625	3.8	0.1490
grep v11	8.9740	85.8181	9.6	0.1942	0.0000	N/A	5.7939
gzip v2	0.0095	0.5625	59.1	113.3410	1.0000	113.3	0.0322
gzip v5	0.0611	15.1111	247.2	64.7306	0.1111	582.6	0.0227
gzip v13	0.0000	2.7000	N/A	109.2140	0.0000	N/A	0.0141
sed v1	0.0095	0.0000	0.0	189.3610	6.1111	31.0	0.0004
sed v3	0.0040	0.2500	63.0	238.7950	91.5000	2.6	0.0062
sed v5	0.3556	31.8333	89.5	12.6217	12.0690	1.0	0.0365
space v19	0.0105	4.6667	444.5	45.7808	13.1667	3.5	0.0057
space v21	0.0000	0.3333	N/A	65.6796	1.0000	65.7	0.0002
space v28	0.0114	23.0000	2016.5	31.2257	26.5000	1.2	0.0016
Average	0.6835	12.9271	1435.9	67.0660	12.2793	73.4	0.4332

0.0611 and 15.1111 failing test cases on gzip v5 pass on m_c and m_f respectively.

Table III provides supporting evidence for the conjectures of MUSE. The number of the failing test cases on P that pass on m_f is 1435.9 times greater than the number on m_c on average, which supports the first conjecture. Similarly, the number of the passing test cases on P that fail on m_c is 73.4 times greater than the number on m_f on average, which supports the second conjecture. Based on the results, we claim that both conjectures are true.

C. Regarding RQ2: Precision of MUSE in terms of the % of executed statements examined to localize a fault

Table IV presents the precision evaluation of Jaccard, Ochiai, Op2, and MUSE with the proportion of executed statements required to be examined before localizing the fault (i.e. the Expense metric). The most precise results are marked in bold. Following the ranking produced by

MUSE, one can localize a fault after examining 0.38% of the executed statements on average. The average precision of MUSE is 25.68 ($=9.67/0.38$), 24.61 ($=9.27/0.38$), and 20.09 ($=7.56/0.38$) times higher than that of Jaccard, Ochiai, and Op2, respectively. In addition, MUSE produces the most precise results for 11 out of the 14 studied faulty versions. This provides quantitative answer to **RQ2**: MUSE can outperform the state-of-the-art SBFL techniques over the Expense metric.

In response to Parnin and Orso [22], we also report the absolute rankings produced by MUSE, i.e. the actual number of statements that need to be inspected before encountering the faulty statement. MUSE ranks the faulty statements of the seven faulty versions (flex v1, v11, gzip v2, v5, v13, and space v21, v28) at the top and ranks the faulty statement of another three versions (flex v7, sed v3, and space v19) among the top three. On average, MUSE ranks the faulty statement among the top 7.43 places, which is 24.86 ($=184.64/7.43$) times more precise than the best performing SBFL technique, Op2. We believe MUSE is precise enough that its results can be used by a human developer in practice.

D. Regarding RQ3: Precision of MUSE in terms of the Locality Information Loss

The Locality Information Loss column of Table IV shows the precision of Jaccard, Ochiai, Op2, and MUSE in terms of the LIL metric, calculated with $\epsilon = 10^{-17}$. The best results (i.e. the lowest values) are marked in bold. The LIL metric value of MUSE is 2.87 on average, which is 1.99 ($=5.72/2.87$), 2.10 ($=6.03/2.87$), and 2.29 ($=6.58/2.87$) times more precise than those of Jaccard, Ochiai, and Op2. In addition, the LIL metric values of MUSE are the smallest ones on the eleven out of the 14 subject program versions.

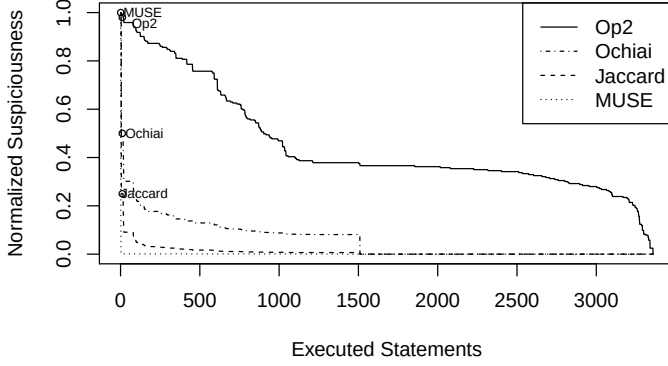


Figure 3: Normalized suspiciousness scores from `space v21` in descending order

This answers **RQ3**: MUSE can outperform the state-of-the-art SBFL techniques over the LIL metric.

One interesting observation is that MUSE produces Expense and LIL values that correlate relatively well. The versions whose absolute ranking of faulty statement is equal to or less than 3, and whose LIL metric is less than or equal to 2.66, are the following 10 versions: `flex v1, v7, v11`, `gzip v2, v5, v13`, `sed v3`, and `space v19, v21, v28`. For another three versions (`grep v3, v11` and `sed v1`), both the Expense and LIL metric values perform worse than the other techniques, although not significantly.

In contrast, Expense and LIL metric often do not agree with each other for the SBFL techniques. Consider `space v21`: Jaccard, Ochiai, and Op2 produces the same Expense value of 0.45%. However, their LIL values are all different (Jaccard: 4.92 < Ochiai: 5.96 < Op2: 7.34). A similar pattern is observed in other subject versions (`flex v7`, `grep v11`, `gzip v2, v5, v13`, `sed v1, v3`, `space v19, v21`).

Figure 3 illustrates this phenomenon in more detail. It plots the normalized suspiciousness scores for each executed statement of `space v21` in a descending order⁶. The circles indicate the location of the faulty statement. While all techniques assign, to the faulty statement, suspiciousness values that rank near the top, it is the suspiciousness of correct statements that differentiates the techniques. When normalized into $[0, 1]$, MUSE assigns values less than 0.00024 to all correct statements while the SBFL techniques assign values much higher than 0 (e.g., 4.8% of the executed statements are assigned suspiciousness higher than 0.9 by Op2, while 37.2% are assigned values higher than 0.5). Figure 4 presents the distribution of suspiciousness in `space v21` for individual techniques to make it easier to observe the differences. This provides supporting evidence to answer **RQ3**: MUSE does perform better than the state-

⁶The normalized suspiciousness of a statement s in an FL technique τ , $norm_susp_{\tau}(s)$ is computed as $(susp_{\tau}(s) - \min(\tau)) / (\max(\tau) - \min(\tau))$ where $\min(\tau)$ and $\max(\tau)$ is the minimum and maximum observed suspiciousness for all statements [23].

of-the-art SBFL techniques when evaluated using the LIL metric. Figure 4 also intuitively illustrates the strength of the LIL metric over the Expense metric.

This independently confirms the results obtained by Qi et al. [23]. Our new evaluation metric, LIL, confirms the same observation as Qi et al. by assigning Jaccard a lower LIL value of 5.72 than that of Op2, 6.58 (see Section III for more details).

VI. DISCUSSIONS

A. Why does it work well?

As shown in Section V-C and Section V-D, MUSE demonstrates superior precision when compared to the state-of-the-art SBFL techniques. In addition to the finer granularity of statement level, the improvement is also partly because MUSE directly evaluates where (partial) fix can (and cannot) potentially exist instead of predicting the suspiciousness through program spectrum. In a few cases, MUSE actually finds a fix, in a sense that it performs a program mutation that will make all test cases pass (this, in turn, increases the first term in the metric, raising the rank of the location of the mutation). However, in other cases, MUSE finds a *partial* fix, i.e. a mutation that will make only some of previously failing test cases pass. While not as strong as the former case, a partial fix nonetheless captures the chain of control and data dependencies that are relevant to the failure and provides a guidance towards the location of the fault.

B. MUSE and Test Suite Balance

One advantage MUSE has over SBFL is that MUSE is relatively freer from the proportion of passing and failing test cases in a test suite. In contrast, SBFL techniques benefit from having a balanced test suite, and have been augmented by automated test data generation work [5, 12, 15].

MUSE does not require the test suite to have many passing test cases. To illustrate the point, we purposefully calculated MUSE metric without any test cases that passed before mutation (this effectively means that we only use the first term of the metric). On average, MUSE ranked the faulty statement within the top 5.09%, which outperforms SBFL techniques that considered all passing and failing test cases: MUSE is still 1.90 ($=9.67/5.09$), 1.82 ($=9.27/5.09$) and 1.49 ($=7.56/5.09$) times more precise than Jaccard, Ochiai, and Op2 respectively.

Interestingly, MUSE does not require the test suite to have many failing test cases. Considering that previous work [12, 15] focused on producing more failing test cases to improve the precision, this is an important observation. We purposefully calculated MUSE metric without any test cases that failed before mutation: although this translates into an unlikely use case scenario, it allows us to measure the differentiating power of the second conjecture in isolation. When only the second term of the MUSE metric is calculated (with $\alpha=1$), MUSE could still rank the faulty statement

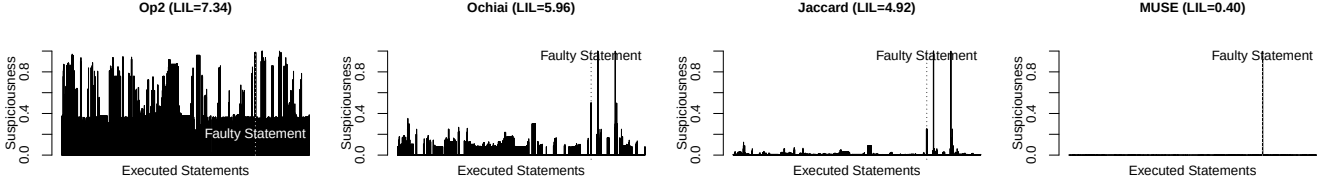


Figure 4: Comparison of distributions of normalized suspiciousness score across executed statements of `space v21`

Table V: Expense, LIL, and NCP scores on `look utx 4.3`

FL Technique	% of executed stmts examined	Locality Information Loss	Average NCP over 100 runs
MUSE	11.25	3.52	25.3
Op2	42.50	3.77	31.0
Ochiai	42.50	3.83	32.2
Jaccard	42.50	3.89	35.5

among the top 14.62% on average, and among the top 2% for seven out of 14 faulty versions we studied. Intuitively, SBFL techniques require many failing executions to identify where a fault is, whereas MUSE is relatively free from this constraint because it also identifies where a fault *is not*.

This advantage is due to the fact that MUSE utilizes two separate conjectures, each of which is based on the number of failing and passing test cases respectively. Thus, even if a test suite has almost no failing or passing test cases, MUSE can localize a fault precisely.

C. LIL Metric and Automated Bug Repair

LIL metric is better at predicting the performance of an FL technique for automated program repair tools than the traditional ranking model. The fact that the ranking model is not suitable has been demonstrated by Qi et al. [23]. We performed a small case study with the GenProg-FL tool by Qi et al., which is a modification of the original GenProg tool. We applied Jaccard, Ochiai, Op2, and MUSE, to GenProg-FL in order to fix `look utx 4.3`, which is one of the subject programs recently used by Le Goues et al. [7]. GenProg-FL [23] measures the NCP (Number of Candidate Patches generated before a valid patch is found in the repair process) of each FL technique where the suspiciousness score of a statement s is used as the probability to mutate s .

Table V shows the Expense, the LIL and the NCP scores on `look utx 4.3` by MUSE, Op2, Ochiai, and Jaccard. For the case study, we generated 30 failing and 150 passing test cases randomly and used the same experiment parameters as in GenProg-FL [23] (we obtained the average NCP score from 100 runs). Table V demonstrates that the LIL metric is useful to evaluate the effectiveness of an FL technique for the automatic repair of `look utx 4.3` by GenProg-FL: the LIL scores (MUSE : 3.52 < Op2 : 3.77 < Ochiai : 3.83 < Jaccard : 3.89) and the NCP scores (MUSE : 25.3 < Op2 : 31.0 < Ochiai : 32.2 < Jaccard : 35.5) are in agreement.

A small LIL score of a localization technique indicates that the technique can be used to perform more efficient automated program repair. In contrast, the Expense metric values did not provide any information for the three SBFL techniques. We plan to perform a further empirical study to support the claim.

VII. RELATED WORK

The idea of generating *diverse program behaviours* to localize a fault more effectively has been utilized by several studies. For example, Cleve and Zeller [9] search for program states that cause the execution to fail by replacing states of a neighbouring passing execution with those of a failing one. If a passing execution with the replaced states no longer passes, relevant statements of the states are suspected to contain faults. Zhang et al. [34], on the other hand, change branch predicate outcomes of a failing execution at runtime to find suspicious branch predicates. A branch predicate is considered suspicious if the changed branch outcome makes a failing execution pass. Similarly, Jeffrey et al. [11] change the value of a variable in a failing execution with the values with other executions; Chandra et al. [2] simulate possible value changes of a variable in a failing execution through symbolic execution. Those techniques are similar to MUSE in a sense that generating diverse program behaviours to localize faults. However, they either *partially* depend on the conjectures of MUSE (some [2, 11, 34] in particular depend on the first conjecture of MUSE) or rely on a different conjecture [9]. Moreover, MUSE does not require any other infrastructure than a mutation tool, because it *directly* changes program source code to utilize the conjectures (Section IV-B).

Since mutation operators vary significantly in their nature, mutation-based approaches such as MUSE may not yield itself to theoretical analysis as naturally as the spectrum-based ones, for which hierarchy and equivalence relations have been shown with proofs [28]. In the empirical evaluation, however, MUSE outperformed Op2 SBFL metric [19], which is known to be the best SBFL technique.

Yoo showed that risk evaluation formulas for SBFL can be automatically evolved using Genetic Programming (GP) [31]. Some of the evolved formulas were proven to be equivalent to the known best metric, Op2 [29]. While MUSE has been manually designed following human intuition, they can be evolved by GP in a similar fashion.

Papadakis and Le-Traon have used mutation analysis for fault localization [21]. However, instead of measuring the impact of mutation on partial correctness as in MUSE (i.e. the conjecture 1), Papadakis and Le-Traon depend on the similarity between mutants in an attempt to detect unknown faults: variations of existing risk evaluation formulas were used to identify suspicious mutants. Zhang et al. [33], on the other hand, use mutation analysis to identify a fault-inducing commit from a series of developer commits to a source code repository: their intuition is that a mutation at the same location as the faulty commit is likely to result in similar behaviours and results in test cases. Although MUSE shares a similar intuition, we do not rely on tests to exhibit similar behaviour: rather, both of MUSE metrics measure what is the *differences* introduced by the mutation. Given the disruptive nature of the program mutation, we believe MUSE is more robust.

VIII. CONCLUSION AND FUTURE WORK

Based on the conjectures we introduced, MUSE increases the suspiciousness of potentially faulty statements and decreases the suspiciousness of potentially correct statements. The results of empirical evaluation show that MUSE can not only significantly outperform the state-of-the-art SBFL techniques, but also provide a practical fault localization solution. The paper also presents Locality Information Loss, a novel evaluation metric for FL techniques based on information theory. A case study shows that it can be better at predicting the performance of an FL technique for automated program repair. Future work includes in-depth study of different mutation operators. We also plan to apply MUSE to larger subjects such as PHP with multiple test suites. In addition, we will apply the mutation idea to concolic unit testing [30] and concurrent coverage-based testing [24].

ACKNOWLEDGEMENTS

This research was supported by the NRF Mid-career Research Program funded by the MSIP Korea (2012R1A2A2A01046172), the ERC of Excellence Program MSIP/NRF of Korea (Grant NRF-2008-0062609), the MSIP under the ITRC support program (NIPA-2013-H0301-13-5004), and the EPSRC, UK (grant number EP/I010165/1). Also, we thank Shin Hong for valuable discussion on MUSE.

REFERENCES

- [1] T. A. Budd. *Mutation analysis of program test data*. PhD thesis, Yale University, 1980.
- [2] S. Chandra, E. Torlak, S. Barman, and R. Bodík. Angelic debugging. In *ICSE*, 2011.
- [3] V. Dallmeier, C. Lindig, and A. Zeller. Lightweight bug localization with ample. In *AADEBUG*, 2005.
- [4] H. Do, S. Elbaum, and G. Rothermel. Supporting controlled experimentation with testing techniques: An infrastructure and its potential impact. *ESE*, 10(4):405–435, 2005.
- [5] D. Gopinath, R. Zaeem, and S. Khurshid. Improving the effectiveness of spectra-based fault localization using specifications. In *ASE*, 2012.
- [6] C. L. Goues, M. Dewey-Vogt, S. Forrest, and W. Weimer. A systematic study of automated program repair: Fixing 55 out of 105 bugs for \$8 each. In *ICSE*, 2012.
- [7] C. L. Goues, T. Nguyen, S. Forrest, and W. Weimer. Genprog: A generic method for automatic software repair. *TOSEM*, 38(1):54–72, 2012.
- [8] H. Agrawal, R. A. DeMillo, B. Hathaway, W. Hsu, W. Hsu, E. W. Krauser, R. J. Martin, A. P. Mathur, and E. Spafford. Design of mutant operators for the c programming language. Technical Report SERC-TR-120-P, Purdue University, 1989.
- [9] H. Cleve and A. Zeller. Locating causes of program failures. In *ICSE*, 2005.
- [10] P. Jaccard. Étude comparative de la distribution florale dans une portion des Alpes et des Jura. *Bull. Soc. vaud. Sci. nat.*, 37:547–579, 1901.
- [11] D. Jeffrey, N. Gupta, and R. Gupta. Fault localization using value replacement. In *ISSTA*, 2008.
- [12] W. Jin and A. Orso. F3: fault localization for field failures. In *ISSTA*, 2013.
- [13] J. A. Jones and M. J. Harrold. Empirical evaluation of the tarantula automatic fault-localization technique. In *ASE*, 2005.
- [14] J. A. Jones, M. J. Harrold, and J. T. Stasko. Visualization for fault localization. In *ICSE*, Software Visualization Workshop, 2001.
- [15] J. Röbler, G. Fraser, A. Zeller, and A. Orso. Isolating failure causes through test case generation. In *ISSTA*, 2012.
- [16] S. Kullback and R. A. Leibler. On information and sufficiency. *Ann. Math. Statist.*, 22(1):79–86, 1951.
- [17] J. C. Maldonado, M. E. Delamaro, S. C. Fabbri, A. da Silva Simão, T. Sugeta, A. M. R. Vincenzi, and P. C. Masiero. Proteum: A family of tools to support specification and program testing based on mutation. In *Mutation testing for the new century*. 2001.
- [18] M. Renieres and S. P. Reiss. Fault localization with nearest neighbor queries. In *ASE*, 2003.
- [19] L. Naish, H. J. Lee, and K. Ramamohanarao. A model for spectra-based software diagnosis. *TOSEM*, 20(3):11:1–11:32, August 2011.
- [20] A. Ochiai. Zoogeographic studies on the soleoid fishes found in Japan and its neighbouring regions. *Bull. Jpn. Soc. Sci. Fish.*, 22(9):526–530, 1957.
- [21] M. Papadakis and Y. Le-Traon. Using mutants to locate “unknown” faults. In *ICST*, Mutation Workshop, 2012.
- [22] C. Parnin and A. Orso. Are automated debugging techniques actually helping programmers? In *ISSTA*, 2011.
- [23] Y. Qi, X. Mao, Y. Lei, and C. Wang. Using automated program repair for evaluating the effectiveness of fault localization techniques. In *ISSTA*, 2013.
- [24] S. Hong, J. Ahn, S. Park, M. Kim, and M. J. Harrold. Testing concurrent programs to achieve high synchronization coverage. In *ISSTA*, 2012.
- [25] W. Weimer, T. Nguyen, C. L. Goues, and S. Forrest. Automatically finding patches using genetic programming. In *ICSE*, 2009.
- [26] E. Wong and V. Debroy. A survey of software fault localization. Technical Report UTDCS-45-09, University of Texas at Dallas, 2009.
- [27] W. E. Wong, Y. Qi, L. Zhao, and K.-Y. Cai. Effective fault localization using code coverage. In *COMPASAC*, 2007.
- [28] X. Xie, T. Y. Chen, F.-C. Kuo, and B. Xu. A theoretical analysis of the risk evaluation formulas for spectrum-based fault localization. *TOSEM*, 22(4):31, 2013.
- [29] X. Xie, F.-C. Kuo, T. Y. Chen, S. Yoo, and M. Harman. Provably optimal and human-competitive results in sbse for spectrum based fault localisation. In *SSBSE*. 2013.
- [30] Y. Kim, Y. Kim, T. Kim, G. Lee, Y. Jang, and M. Kim. Automated unit testing of large industrial embedded software using concolic testing. In *ASE Experience Track*, 2013.
- [31] S. Yoo. Evolving human competitive spectra-based fault localisation techniques. In *SSBSE*. 2012.
- [32] S. Yoo, M. Harman, and D. Clark. Fault localization prioritization: Comparing information-theoretic and coverage-based approaches. *TOSEM*, 22(3):19:1–19:29, July 2013.
- [33] L. Zhang, L. Zhang, and S. Khurshid. Injecting mechanical faults to localize developer faults for evolving software. In *OOPSLA*, 2013.
- [34] X. Zhang, N. Gupta, and R. Gupta. Locating faults through automated predicate switching. In *ICSE*, 2006.